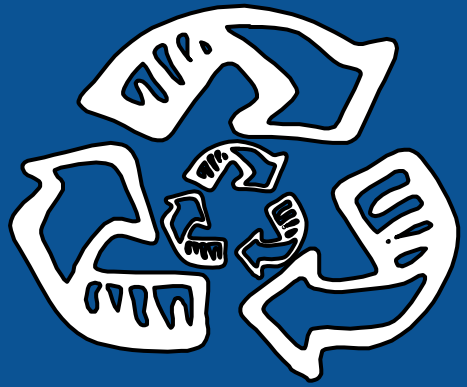
Архитектура ЭВМ и язык ассемблера

Семинар #33:

1. Перемножение матриц:
   - Организация двумерного массива.
   - Вложенные циклы.
2. Кэширование данных:
   - Сложности со сложностью алгоритма.
   - Транспонирование матрицы.

27.02.2026

Пример перемножения матриц



Алгоритм перемножения матриц

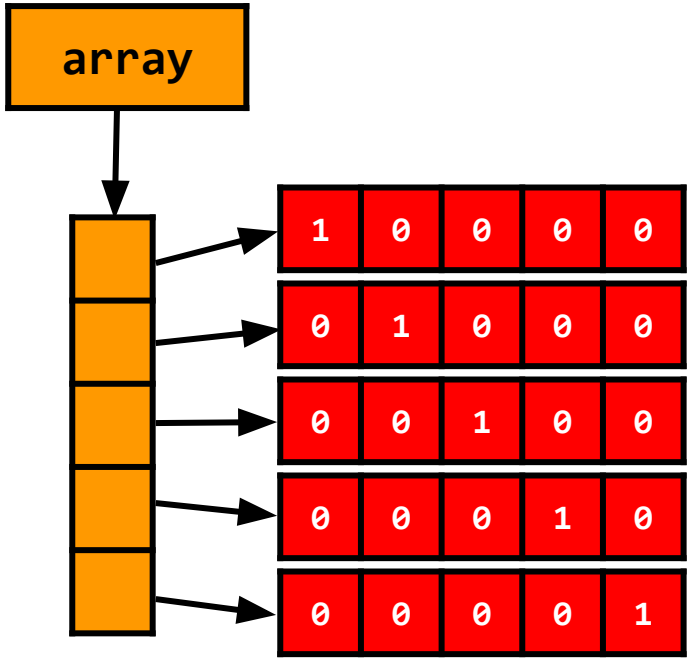
$$(AB)_{ij} = \sum_{k=0}^{N-1} a_{ik} b_{kj} = a_{i1} b_{1j} + a_{i2} b_{2j} + \cdots + a_{i(N-1)} b_{(N-1)j}$$

Реализация на C для квадратных матриц NxN

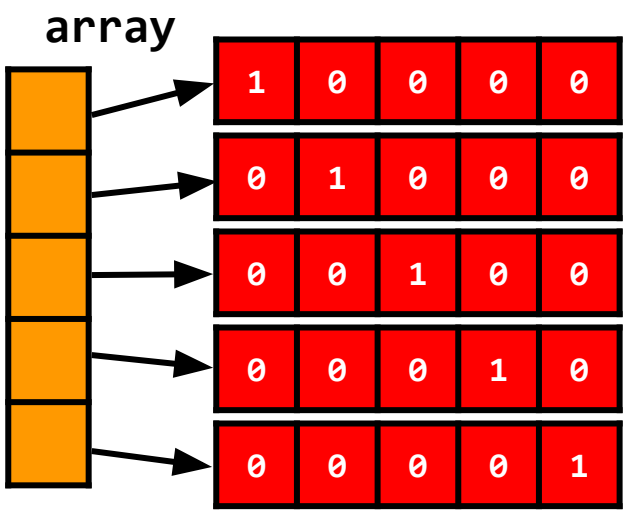
```
uint32_t A[N][N], B[N][N], C[N][N];
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        for (int k = 0; k < N; k++) {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}
```

Размещение двумерного массива в памяти

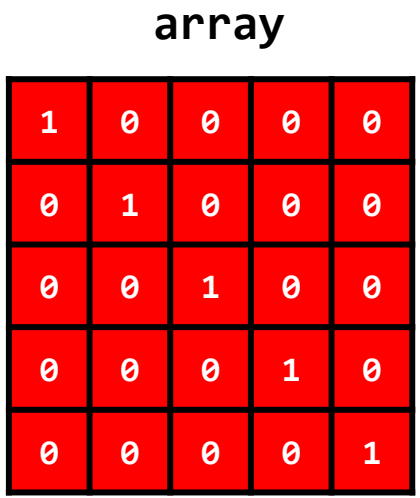
```
int** array;
```




```
int* array[N];
```




```
int array[N][N];
```



Адресация двумерного массива

```
int array2d[N][N];
```

```
int array1d[N*N];
```

array

x

y

1	0	0	0	0
0	1	0	0	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	1

```
array2d[y  ][x  ] ↔ array1d[N*y     + x  ]
array2d[y  ][x+1] ↔ array1d[N*y     + x+1]
array2d[y+1][x  ] ↔ array1d[N*(y+1) + x  ]
```

```
x += 1 ↔ index += 1
y += 1 ↔ index += N
```

Один регистр вместо двух!!!

index

1	0	0	0	0	0	1	0	0	0	0	0	1	0	0	…

Организация вложенного цикла

Вычисление адресов с помощью инструкции LEA

```
lea     edi, dword [mat_B + 4 * ebx]

imul    esi, eax, 4 * MATRIX_SIZE
add     esi, mat_A
```

Сохранение регистров в память («регистровое давление»)

```
mov     dword [eax_tmp], eax
mov     dword [ebx_tmp], ebx

; <ВЛОЖЕННЫЙ ЦИКЛ>

mov     eax, dword [eax_tmp]
mov     ebx, dword [ebx_tmp]
```

Кэширование данных

Производительность перемножения матриц

Теоретическая оценка

Количество сложений: $N^3 + (N-1)\cdot N^2 + N\cdot(N-1) + (N-1) = O(N^3)$

Количество умножений: $N^3 = O(N^3)$

Анализ инструкций в ASM

Тоже $O(N^3)$

Результаты запуска (Intel(R) Core(TM) i7-4770HQ 2.2 ГГц)

N	256	400	512	800	1024	1600	2048	4096
Память	256 КБ	625 КБ	1 МБ	2.4 МБ	4 МБ	9.8 МБ	16 МБ	64 МБ
T, с	0.03	0.06	0.22	0.63	2.5	31	64	612
T/N^3, нс	1.7	0.9	1.6	1.23	2.3	**7.6**	**7.5**	**8.9**

Идея кеширования

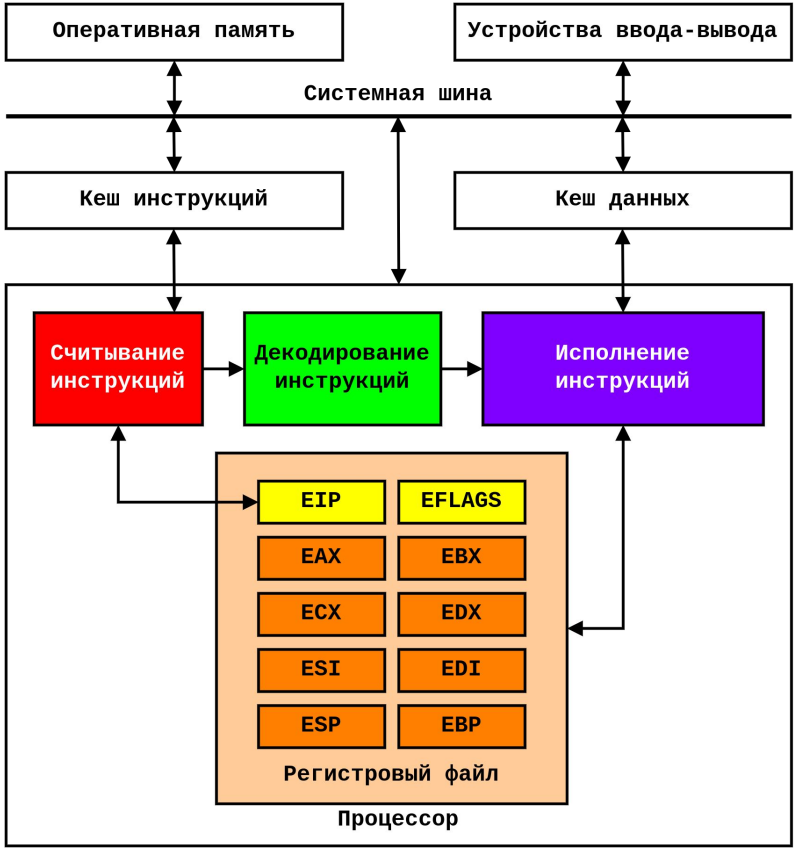
В кеше хранятся недавно использованные ячейки памяти.

Запись:

- Выполняется в кэш данных.
- Рано или поздно попадёт в память.

Чтение:

- Сначала пробуем читать из кэша.
- Если ячейка отсутствует в кэше, идём в оперативную память.



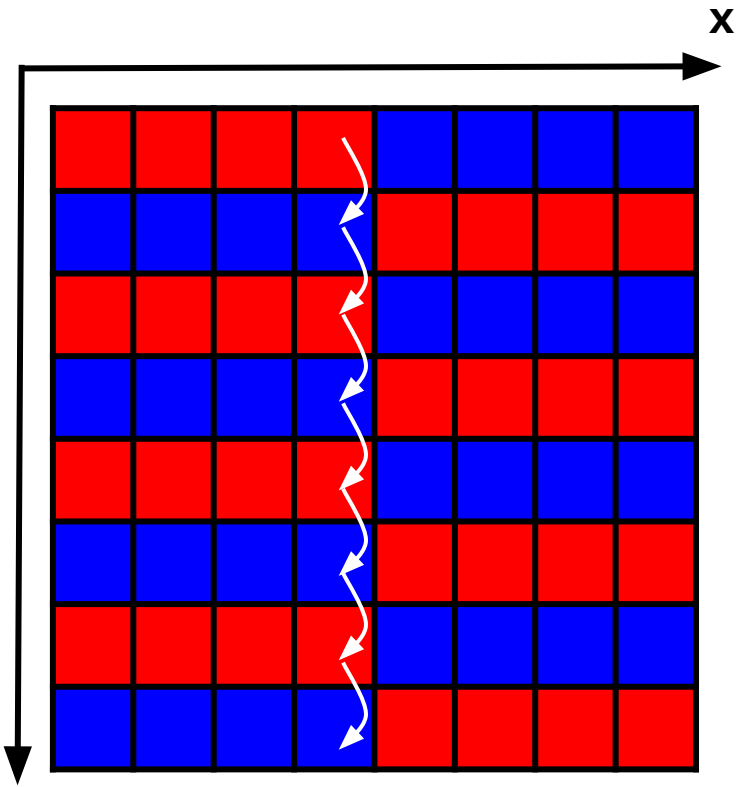
Транспонирование матрицы

$$(AB)_{ij} = \sum_{k=0}^{N-1} a_{ik} b^{\mathrm{T}}_{jk} = a_{i1} b^{\mathrm{T}}_{j1} + a_{i2} b^{\mathrm{T}}_{j2} + \cdots + a_{i(N-1)} b^{\mathrm{T}}_{j(N-1)}$$

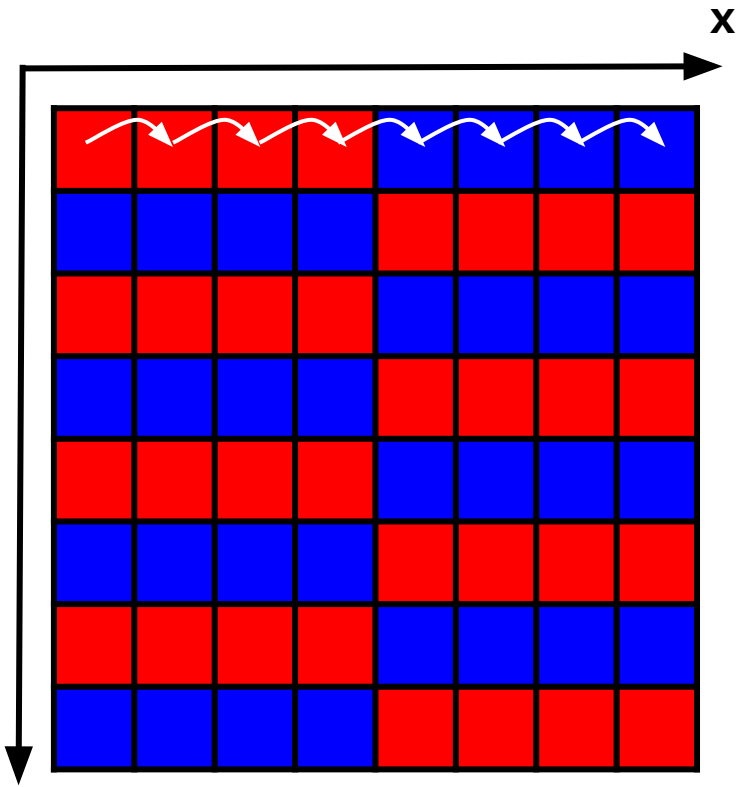
```
uint32_t A[N][N], B[N][N], C[N][N];

matrix_transpose(B);
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        for (int k = 0; k < N; k++) {
            C[i][j] += A[i][k] * B[j][k];
        }
    }
}
```

Последовательность доступов в память

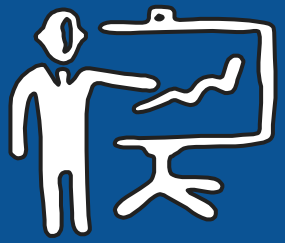


8 кеш-линий



2 кеш-линии

Вопросы?



Красивые иконки взяты с сайта handdrawngoods.com